

Assignment 1: Epidemic rate equations

Rasmus (aka. C++mus), Asger, Benjamin

February 7, 2026

1 Implement and validate the SIR model

After implementing the SIR-model into C++, we test our implementation by considering different limits for our parameters. We used Euler integration to solve the set of differential equations numerically. We have chosen a timestep where there is no visual difference between the chosen timestep (in this case $\Delta t = 0.01$) and the time step divided by 10.

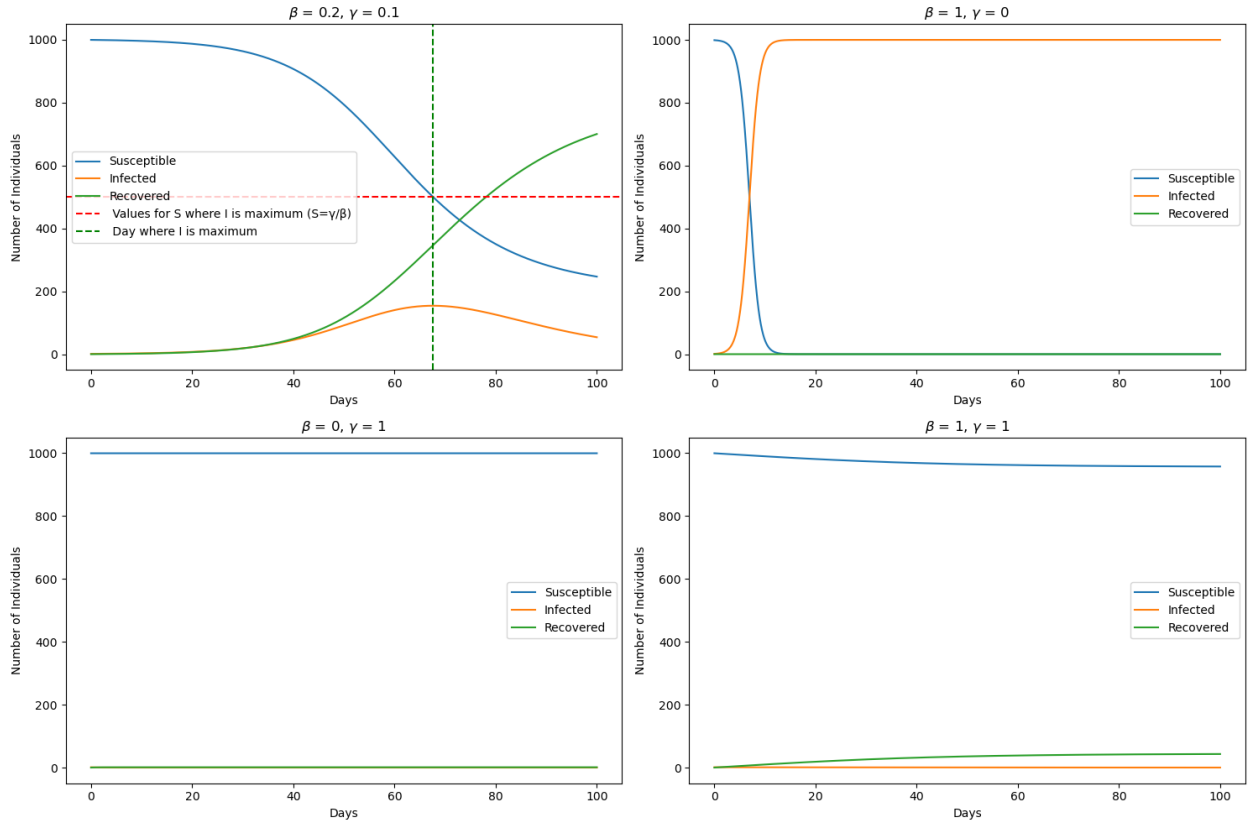


Figure 1: SIR simulation runs with different parameter choices: Standard choice $\gamma = 0.1, \beta = 0.2$ (Top Left), high infectivity with low recovery rate $\beta/\gamma \rightarrow \infty$ (Top right), low infectivity $\beta/\gamma \rightarrow 0$ (Bottom left) and equal infectivity and recovery rates $\beta/\gamma \rightarrow 1$ (Bottom right).

These tests serve as methods of validation for our model and implementation. In the top left of figure 1 we compare our simulation against an analytical result. The maximum number of infected occurs at the time where $S = \frac{\gamma}{\beta}$. We see that that our simulation is in great agreement with this analytical result.

Looking at the case where $\beta = 1, \gamma = 0, \frac{\beta}{\gamma} = \infty$, we see that the number of people infected explodes in the start, since there is no recovery rate to suppress the rapid growth. Also as expected S and I are

mirrored around 500, since S decreases with the same rate as I increases. Looking at the reverse case where $\beta = 0$, $\gamma = 1$, $\frac{\beta}{\gamma} = 0$, nothing really happens, as the one infected recovers, and there is no way to produce more infected.

For the last case where $\beta = 1$, $\gamma = 1$, $\frac{\beta}{\gamma} = 1$, we see that I never becomes significantly large and therefore R doesn't become larger than S . Comparing it to the $\beta = 1$, $\gamma = 0$ case, we see that the value of γ has a significant dampening effect on the spread of the infection, since for both simulations we used the same β -value, but in one of them the infection becomes much more widespread.

2 Use and plot the SIR mode

We now simulate our model with $\beta = 0.2$ and $\gamma = 0.1$ using differently sized timesteps. Again we choose the small timestep to be where there is no visual difference between the chosen timestep (in this case $\Delta t = 0.01$) and the time step divided by 10. The large timestep we chose such that we could see the discreteness of the algorithm.

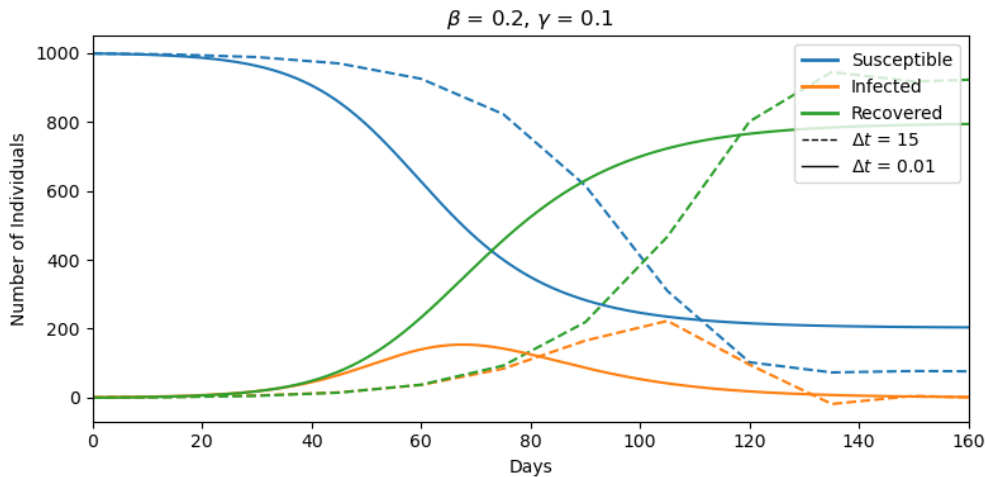


Figure 2: SIR simulation runs with parameters $\beta = 0.2$ and $\gamma = 0.1$. The striped curves are obtained through Euler integration with a large timestep ($\Delta t = 15$ days), while the continuous curves were calculated using significantly smaller timestep ($\Delta t = 0.01$ days).

3 Timestep convergence

To test the importance of the chosen timestep, we calculate the error for $\Delta t \in [10^{-6}, 1]$, using the formula:

$$\text{err} = \max_t (S^{\Delta t}(t) - S^{\frac{\Delta t}{2}}(t), I^{\Delta t}(t) - I^{\frac{\Delta t}{2}}(t), R^{\Delta t}(t) - R^{\frac{\Delta t}{2}}(t))$$

I.e. for each Δt we run two simulations: One with timestep Δt and the other with timestep $\Delta t/2$. We then subtract every second value for the variables $S(t)$, $I(t)$, $R(t)$ of the latter simulation, from that of the former. In figure 3 to the left, we have plotted the maximum error. We ran two simulations, one where we used floats to store our values and one where we used doubles. We see that when going to small Δt for the single precision runs there is probably some rounding errors as our variables are not built to handle the precision we want. When using doubles, we see there is no problem and the error decreases linearly, which might be expected since Euler integration is linear in Δt . In figure 3 to the right, we see the maximal error over time for two different time steps. Note that there are two y axes, but that the shape is almost the same even a few order of magnitude between, but slightly shifted. Interestingly we see that there is a kink, where the maximal error goes from being in S to being in R .

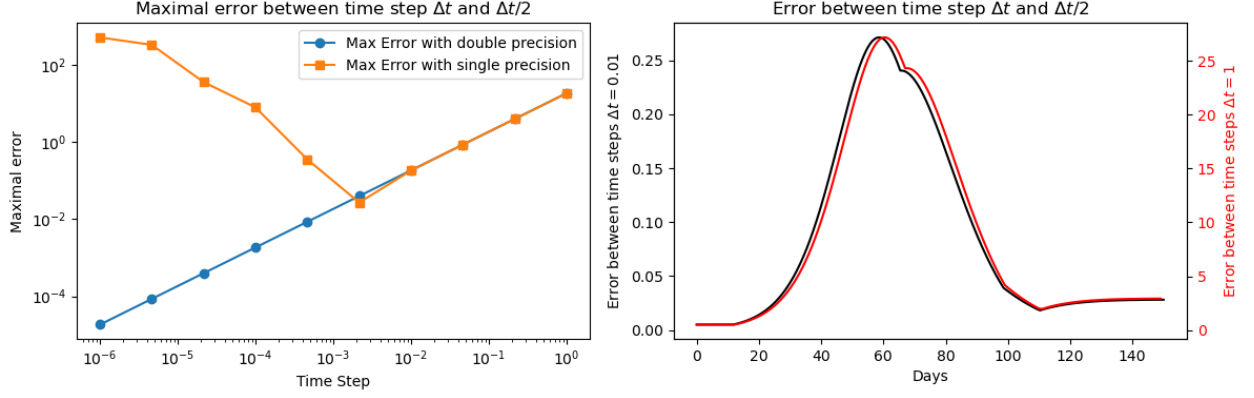


Figure 3: Error calculation for timesteps $\Delta t \in [10^{-6}, 1]$.

4 Integrate C++ with Python

We use the pybind11 library to be able to call our c++ code. In addition we create a function that returns the simulation as a numpy array to be able to use curve_fit. To get the exact same times as in the dataset we use np.interp to linearly interpolate. We use a timestep of $\Delta t = 0.001$ days. In figure 4 we see the data indicated by dots, and the fit indicated by the lines. We see the data fits the model quite well. We don't have errors, but we can see the deviation between the model and data are largest when the value of S and R are largest, indicating some sort of Poisson like errors or at least errors that are dependent on the number of individuals. We get parameters $\gamma = 0.0736$ and $\beta = 0.2451$.

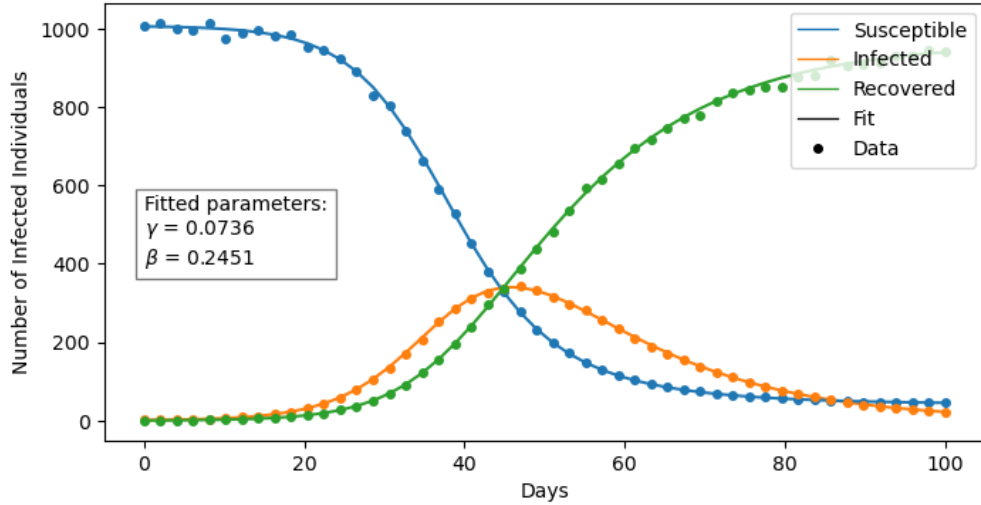


Figure 4: Fit of the SIR model to the data.

5 Appendix

So we have two different codes. One is the c++ implementation, which is not the same as the one which we call from Python. The codes have sort of evolved in different directions, so they work in slightly different ways

5.1 SIR.cpp

```
#include <iostream>
#include <fstream>
#include <vector>

// Function to take a step in the SIR model
// state: vector of S, I, R
// beta: infection rate
// gamma: recovery rate
// dt: time step
// N: the total number of people
std::vector<double> take_step(std::vector<double> state, double beta, double gamma, double dt, double N){
    // We define the derivatives and multiply by the change in time (Simple euler integration)
    double dS = -beta * state[1] * (state[0]/N) * dt;
    double dI = (beta * state[1] * (state[0]/N) - gamma * state[1]) * dt;
    double dR = gamma * state[1] * dt;
    double new_S = state[0] + dS;
    double new_I = state[1] + dI;
    double new_R = state[2] + dR;
    std::vector<double> new_state = { new_S, new_I, new_R };

    return new_state;
}

// Saves data to a csv
void save_data_to_csv(std::vector<std::vector<double>> data, char* file_name)
{
    std::ofstream data_file;
    data_file.open(file_name);
    int number_of_steps = data.size();
    for (int i = 0; i < number_of_steps; i++)
    {
        data_file << data[i][0] << "," << data[i][1] << "," << data[i][2] << "," << data[i][3] << "\n";
    }
    data_file.close();
}

// Simulation loop
std::vector<std::vector<double>> simulate(std::vector<double> initial_frame, std::vector<double> parameters)
{
    double N = parameters[0];
    double beta = parameters[1];
    double gamma = parameters[2];
    double dt = parameters[3];
    double number_of_days = parameters[4];

    int number_of_steps = number_of_days / dt;
    std::vector<std::vector<double>> history(number_of_steps);
    history[0] = { 0, initial_frame[0], initial_frame[1], initial_frame[2] };
    std::vector<double> state;
    std::vector<double> new_state;
    for (int i = 0; i < number_of_steps-1; i++)
    {
        state = { history[i][1], history[i][2], history[i][3] };
        new_state = take_step(state, beta, gamma, dt, N);
        history[i+1] = { dt*i, new_state[0], new_state[1], new_state[2] };
    }
    return history;
}

// Takes in a matrix and outputs the largest value in that matrix
double max(std::vector<std::vector<double>> matrix)
{
    double maximum = 0; // assuming some values a positive
    int length = matrix.size();
    for (int i = 0; i < length; i++){
        for (int j = 0; j < 3; j++){
            if (matrix[i][j] > maximum){maximum = matrix[i][j];}
        }
    }
}
```

```

        return maximum;
    }

std::vector<double> max_vector(std::vector<std::vector<double>> matrix)
{
    double max1 = 0; // assuming some values a positive
    double max2 = 0;
    double max3 = 0;
    int length = matrix.size();
    for (int i = 0; i < length; i++){
        if (matrix[i][0] > max1){max1 = matrix[i][0];}
        if (matrix[i][1] > max1){max1 = matrix[i][1];}
        if (matrix[i][2] > max1){max1 = matrix[i][2];}
    }
    std::vector<double> maximum = { max1, max2, max3 };
    return maximum;
}

// Takes two matrixes, the first is the smallest, then it only find the differences on the same commulative time step
std::vector<std::vector<double>> difference(std::vector<std::vector<double>> vector1, std::vector<std::vector<double>> vector2)
{
    int length = vector1.size()-1;
    std::vector<std::vector<double>> diff(length);
    for (int i = 0; i < length; i++)
    {
        std::cout << vector1[i+1][2] << " - " << vector2[(i)*2+1][2] << " = " << (vector1[i+1][2] - vector2[(i)*2+1][2]);
        diff[i] = { vector1[i+1][1] - vector2[(i)*2+1][1], vector1[i+1][2] - vector2[(i)*2+1][2], vector1[i+1][3] - vector2[(i)*2+1][3] };
    }
    return diff;
}

// Error computation function
std::vector<std::vector<double>> error(std::vector<double> initial, std::vector<double> params, std::vector<double> DT)
{
    // Run simulation for all the timesteps in DT and the timesteps that are halved
    int length = DT.size();
    std::vector<std::vector<double>> results(length);
    for (int i = 0; i < length; i++){
        double dt1 = DT[i];
        double dt2 = DT[i]/2;
        params[3] = dt1;
        std::vector<std::vector<double>> history1 = simulate(initial, params);
        save_data_to_csv(history1, "history1.csv");
        params[3] = dt2;
        std::vector<std::vector<double>> history2 = simulate(initial, params);
        save_data_to_csv(history2, "history2.csv");

        std::vector<std::vector<double>> difference_matrix = difference(history1, history2);

        double maximum = max(difference_matrix);

        results[i] = {dt1, max};
    }
    return results;
}

//===== Main function =====
int main(int argc, char* argv[]){
    // All the params needed
    std::vector<double> start_state = { 1000, 1, 0 }; // (S, I, R)
    std::vector<double> parameters = { 1000, 0.2, 0.1, 0.1, 2 }; // (N, beta, gamma, dt, #_of_days)

    // A simple for loop for implementation will do just fine
    std::vector<std::vector<double>> history = simulate(start_state, parameters);

    //=====Error generation=====
    std::vector<double> delta_times = { 0.000001, 0.000004, 0.000008, 0.00001, 0.00002, 0.00004, 0.00008, 0.0001, 0.0002 };
    std::vector<std::vector<double>> err = error(start_state, parameters, delta_times);

    // Saving in a csv file (look at the function if it is of interest)
    save_data_to_csv(err, "Error.csv");
}

```

5.2 SIR_Python.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <pybind11/pybind11.h>
#include <cmath>
#include <vector>
#include <pybind11/numpy.h>

namespace py = pybind11;

using namespace std;

vector<double> take_step(vector<double> state, double beta, double gamma, int N, double dt ){
    double dS = -beta * state[1] * state[0]/N;
    double dI = beta * state[1]* state[0]/N - gamma * state[1];
    double dR = gamma * state[1];
    state[0] = state[0] + dS * dt;
    state[1] = state[1] + dI * dt;
    state[2] = state[2] + dR * dt;
    return state;
}

void SIR_savedata(double beta, double gamma, int N, double dt, double tend, string dataname){
    double t = 0;
    vector<double> state = {N-1.0f, 1.0, 0.0};

    ofstream outfile(dataname);
    outfile << "Time,S,I,R,N-\n";
    outfile << t << ", " << state[0] << ", " << state[1] << ", " << state[2] << "\n";

    outfile.close();
    outfile.open(dataname, ios_base::app);

    int steps = static_cast<int>(std::floor(tend / dt));

    for (int i = 1; i < steps; i++){
        state = take_step(state, beta, gamma, N, dt);
        t = i * dt;
        outfile << t << ", " << state[0] << ", " << state[1] << ", " << state[2] << "\n";
    }
    outfile.close();
}

py::array_t<double> SIR_returndata(double beta, double gamma, int N, double dt, double tend){
    vector<double> state = {N-1.0, 1.0, 0.0};

    int steps = static_cast<int>(std::floor(tend / dt));
    py::array_t<double> data_matrix({steps, 4});
    auto buf = data_matrix.mutable_unchecked<2>();
    buf(0,0) = 0;
    buf(0,1) = state[0];
    buf(0,2) = state[1];
    buf(0,3) = state[2];

    for (int i = 1; i < steps; i++){
        state = take_step(state, beta, gamma, N, dt);
        buf(i,0) = i*dt;
        buf(i,1) = state[0];
        buf(i,2) = state[1];
        buf(i,3) = state[2];
    }
    return data_matrix;
}

py::array_t<double> SIR_errors(double beta, double gamma, int N, double dt, double tend){
    vector<double> state1 = {N-1.0, 1.0, 0.0};

    vector<double> state2 = {N-1.0, 1.0, 0.0};

    py::array_t<double> max_diff(3);
    auto buf = max_diff.mutable_unchecked<1>();

    buf(0) = 0.0;
    buf(1) = 0.0;
```

```

buf(2) = 0.0;

double diff_S;
double diff_I;
double diff_R;
int steps = static_cast<int>(std::floor(tend / dt));

for (int i = 0; i < steps; i++){

    state1 = take_step(state1, beta, gamma, N, dt);

    for (int k=0; k < 2; k++){
        state2 = take_step(state2, beta, gamma, N, dt/2);
    }
    if (i > 5){
        diff_S = std::abs(state1[0] - state2[0]);
        diff_I = std::abs(state1[1] - state2[1]);
        diff_R = std::abs(state1[2] - state2[2]);
        if (diff_S > buf(0)){
            buf(0) = diff_S;
        }
        if (diff_I > buf(1)){
            buf(1) = diff_I;
        }
        if (diff_R > buf(2)){
            buf(2) = diff_R;
        }
    }
}

return max_diff;
}

PYBIND11_MODULE(SIR_module, m) {
    m.doc() = "Performs Euler integration on the SIR model."; // optional docstring
    m.def("SIR_savedata", &SIR_savedata, "Calculates SIR and saves it as a file");
    m.def("SIR_returndata", &SIR_returndata, "Calculates SIR, return a numpy array");
    m.def("SIR_errors", &SIR_errors, "Calculates the max errors for SIR between dt and dt/2");
}

//python SIR_setup.py build_ext --inplace

```